

HRCF: A Hybrid Recursive–Folding Framework for Long-Horizon Reasoning in Large Language Models Beyond the Context Window

Manikanta Reddy Mandadhi, *Independent Researcher*

Abstract

Large language model (LLM) agents tackling deep research, repository-scale software engineering, and multi-day automation pipelines are now constrained less by raw model capability than by the engineering of the context they consume. Two distinct paradigms have emerged in 2025–2026 to address this bottleneck. *Recursive Language Models* (RLMs) [1], [2] treat the prompt as a Python variable inside a REPL and let the root model dispatch sub-LLM calls over arbitrary slices of it, breaking the linear coupling between input length and effective context. *Context-Folding* [4]–[6] operates orthogonally: the agent itself *branches* into sub-trajectories, summarizes them on *return*, and folds intermediate steps out of working memory. Both paradigms outperform retrieval-augmented and summarization baselines on long-horizon benchmarks, but each has structural blind spots: RLMs excel at *static* long inputs but pay round-trip latency for every sub-call; folding agents excel at *dynamic* long trajectories but lose globally relevant detail at fold points.

We propose **HRCF (Hybrid Recursive-Folding)**, a unified inference-time framework that subsumes both regimes under a single control policy. HRCF maintains an explicit *context topology*, a directed acyclic graph of context fragments labelled by access pattern (read-only static, write-heavy dynamic, or mixed); and dispatches each fragment to the appropriate handler: RLM-style programmatic delegation for static fragments, fold-branch-return for dynamic ones, and a novel *anchored fold* primitive for mixed fragments that retains regex-addressable handles into folded summaries. We provide a formal cost model for each primitive, prove a competitive ratio bound against the per-fragment optimal policy, and instantiate HRCF in a 1,400-line Python reference implementation evaluated on (i) a synthetic multi-document needle-in-haystack benchmark stressing inputs up to 2×10^6 tokens, (ii) a re-implementation of the OOLONG-Pairs aggregation task [7], and (iii) a long-horizon SWE simulation harness derived from the public Context-Folding release [4]. On all three, HRCF matches or exceeds the stronger of pure-RLM and pure-folding baselines while reducing total token consumption by 27–41 % and wall-clock latency by 18–34 % relative to the best non-hybrid configuration. Ablations isolate the contribution of the topology classifier (most impactful), the anchored-fold primitive, and the fragment-level cost model. We discuss connections to inference-time scaling [8], recursive self-improvement [9], and the broader research programme of teaching language models to manage their own context [2], and we release reference code and benchmark harnesses to support reproduction.

Index Terms

M. R. Mandadhi is an Independent Researcher (e-mail: manikantareddymandadhi@my.unt.edu).

ORCID: 0009-0008-6139-7685. Code and supplementary materials available at <https://github.com/Mani9006>.

This research was self-funded by the author. The author declares no competing interests.

Manuscript prepared April 26, 2026.

Large language models, long-context reasoning, recursive language models, context folding, inference-time scaling, agentic systems, context engineering, hybrid frameworks.

I. INTRODUCTION

THE dominant narrative around large language model (LLM) progress in 2024 was a race for raw context length: from 8K tokens in early GPT-4 deployments to one-million-token windows in Gemini 1.5 Pro and Claude 3 deployments by late 2024 [13], [14]. By mid-2025, however, a different consensus had crystallized in the agentic-systems community. Even when models nominally support million-token windows, three structural limits persist. First, attention scales quadratically with sequence length and effective recall *degrades* sharply past a few hundred thousand tokens; the *context-rot* [12] or *lost-in-the-middle* effect [12]. Second, every additional token in context is paid for at every decoding step, making long-context inference economically untenable for high-throughput deployment [15]. Third, and most fundamentally, the *linear* accumulation of trajectory state in a long-horizon agent loop is incompatible with the exponential growth of accomplishable task length: METR has documented that the length of tasks frontier models can reliably complete is doubling every ≈ 7 months [16], while context windows grow only an order of magnitude per generation.

In response, two architectural paradigms have emerged that displace context management out of the model’s attention mechanism and into the inference scaffold itself.

Recursive Language Models (RLMs). Introduced by Zhang as a blog post in October 2025 [3] and formalized in the *Recursive Language Models* preprint [1], an RLM exposes the same external interface as a vanilla LLM, a `prompt` $\rightarrow$ `response` string mapping, but internally the prompt is bound to a Python variable inside a persistent REPL. The root LLM (depth 0) is shown only the variable name and metadata (length, type), and is given two affordances: arbitrary Python execution against the variable, and the ability to invoke sub-LLMs (depth 1) on any string slice. Prime Intellect adopted this paradigm in early 2026 [2], branding it “the paradigm of 2026” and integrating it into their `verifiers` and `prime-rl` training stacks via the `RLMenv` environment [11]. Empirically, RLMs handle inputs up to two orders of magnitude beyond the underlying model’s context window and, on the OOLONG-Pairs task [7], an RLM driven by GPT-5-mini more than doubles the accuracy of vanilla GPT-5 at lower cost per query [1].

Context Folding. Concurrently and independently, Sun *et al.* [4] formulated *context folding* as an agentic mechanism in which a long-horizon trajectory is dynamically segmented into branches: the agent branches into a sub-trajectory that retains the full preceding context, and on `return` only a self-chosen summary of the branch is preserved in the parent context; the intermediate steps are “folded out”. Sun *et al.* train this behaviour end-to-end via a process-rewarded RL algorithm called *FoldPO* and demonstrate $10\times$ smaller active context with no quality loss on Deep Research and SWE-bench-derived tasks. Wang *et al.* [5] extend this to AgentFold, which uses rejection-sampled fine-tuning on Qwen3-30B-A3B for proactive intra-task folding. Zhao *et al.* [6] introduce FoldAct, an RL formulation for stability under self-conditioning during folded rollouts.

These two paradigms are commonly discussed in the same breath, Prime Intellect explicitly frames RLM as “a way to do context folding” [2], but they are in fact *architecturally orthogonal*. RLMs operate on a *static* long input that the agent passively examines; folding operates on a *dynamic* long trajectory the agent itself produces. Their failure modes are correspondingly disjoint:

- RLMs incur a sub-LLM round-trip latency for every targeted slice, which dominates wall-clock cost when the relevant fragments are small, numerous, and conditionally interdependent (the access pattern of a debugging trace).

- Folding loses globally addressable detail at fold points: once a branch has been summarized, only the summary is queryable, and any later step that needs a verbatim quote from the branch must either re-execute the branch or accept the summary’s compression error.

No prior work, to our knowledge, has unified the two regimes under a common control policy that decides *per-fragment* which primitive to apply.

A. Our Contributions

This paper introduces **HRCF (Hybrid Recursive-Folding)**, a hybrid inference-time framework that subsumes RLM-style recursive delegation and trajectory-level folding under a single, fragment-level control policy. Concretely:

- 1) **Context topology formalism (Section III)**. We model an agent’s working set as a directed acyclic graph $\mathcal{T} = (V, E)$ of *context fragments*, each labelled by an access-pattern descriptor $\pi \in \{\text{STATIC}, \text{DYNAMIC}, \text{MIXED}\}$. The descriptor captures whether the fragment is read-only after instantiation, append-only, or both read and re-read with intervening writes. The labelling is produced by a lightweight learned classifier whose features include token length, write-arc count, and a syntactic-domain prior.
- 2) **Three context primitives with a unified cost model (Section IV)**. HRCF dispatches fragments to one of three primitives: R-DELEGATE (RLM-style sub-LLM call against a Python-bound variable), F-BRANCHRETURN (Sun-style branch-summarize-fold), or A-FOLD (*anchored fold*, our novel primitive that retains regex-addressable handles into the folded summary). We give an analytical cost model for each primitive in terms of token, latency, and information-loss components, and a per-fragment dispatch rule that minimizes expected cost subject to a recall constraint.
- 3) **Competitive-ratio analysis (Section V)**. Under mild assumptions on the cost model, HRCF’s per-fragment dispatch is $O(\log d)$ -competitive against the offline per-fragment optimal policy, where d is the maximum recursion depth. We prove this via a potential-function argument and a reduction to weighted-caching with eviction [23], [24].
- 4) **Anchored fold primitive (Section VI)**. A-FOLD is a novel summarization scheme that, in addition to a natural-language summary, emits a structured handle table: $\langle \text{regex pattern}, \text{byte-offset range}, \text{semantic label} \rangle$ triples that allow a later step to re-extract verbatim spans from the original branch trajectory at zero LLM cost. We show that for trajectories with $\rho < 0.4$ *global re-reference rate*, A-FOLD matches F-BRANCHRETURN in token cost while improving downstream accuracy by 6–14 % on synthetic evaluations.
- 5) **Reference implementation and three-benchmark evaluation (Sections VII–IX)**. We release a 1,400-line Python implementation of HRCF that wraps the `rlm` [10] and Context-Folding [4] reference codebases. We evaluate on (i) a synthetic Multi-Document Needle-in-a-Haystack (MD-NIAH) benchmark with inputs up to 2×10^6 tokens; (ii) the OOLONG-Pairs aggregation task [7]; and (iii) a long-horizon SWE simulation harness derived from the public Context-Folding release. HRCF matches or exceeds the stronger of pure-RLM and pure-folding on all three, while reducing total token consumption by 27–41 % and wall-clock latency by 18–34 % relative to the best non-hybrid configuration.
- 6) **Ablations and analysis (Section X)**. We isolate the contribution of (a) the topology classifier (largest single contribution; removing it costs 11.4 percentage points), (b) the anchored-fold primitive (4.7 pp), and (c) the per-fragment cost model (3.1 pp). We additionally report negative results: at very short input lengths ($< 32\text{K}$ tokens), HRCF’s overhead is not amortized and pure direct-prompt outperforms by 2–3 pp.

- 7) **Discussion of training implications (Section XIV).** HRCF is fundamentally an inference-time scaffold, but; following Zhang *et al.*'s observation that RLM-Qwen3-8B post-trained natively on the RLM scaffold outperforms vanilla Qwen3-8B by 28.3 % on average [1], we discuss how the same RL-with-process-rewards programme used for FoldPO [4] could be extended to train models natively as HRCF policies, and present preliminary evidence from a 200-trajectory supervised fine-tuning run.

B. Why this matters now

Context engineering [33], [34] has moved from a folk-tradition collection of prompt tricks to a recognized research subdiscipline. The ICLR 2026 Workshop on AI with Recursive Self-Improvement [9] explicitly lists *inference-time scaling* and *context management* among its five core lenses. Production agentic systems, from Claude Code's session-level summarization [17] to OpenAI Codex's file-based scaffolding to ByteDance's deep research agents [5], are all converging on some flavour of context delegation, but the design space is still being mapped empirically rather than theoretically. We see HRCF as a step toward that theory: a framework in which the choice between recursive delegation and trajectory folding is not a system-design dichotomy but a per-fragment decision under an explicit cost model.

The remainder of this paper is organized as follows. Section II gives a self-contained background on RLMs, context folding, and adjacent context-management literature. Section III introduces the context-topology formalism. Section IV defines the three primitives and their cost model. Section V presents the competitive-ratio analysis. Sections VI and VII detail the anchored-fold primitive and the reference implementation. Sections VIII–X report empirical results. Section XI situates HRCF in the broader landscape; Section XIV discusses limitations, broader implications, and open problems; Section XVI concludes.

II. BACKGROUND AND PRELIMINARIES

A. Notation

Let $\mathcal{M} : \mathcal{V}^* \rightarrow \mathcal{V}^*$ denote a base autoregressive language model that maps a token sequence (over vocabulary $\mathcal{V}$) to a token sequence, with effective context window W and per-token decoding cost c_{dec} . Let $C \in \mathcal{V}^*$ denote the *full* context the agent must reason over, with $|C| = N$, possibly $N \gg W$. We are interested in inference-time strategies S that compute an answer $a = S(\mathcal{M}, q, C)$ to a query q while accessing $\mathcal{M}$ only through finite-length completions.

The *strategy cost* decomposes as

$$\text{Cost}(S) = \alpha \cdot \text{Tok}(S) + \beta \cdot \text{Lat}(S) + \gamma \cdot \text{Loss}(S), \quad (1)$$

where $\text{Tok}(S)$ is total tokens fed to or generated by $\mathcal{M}$, $\text{Lat}(S)$ is wall-clock latency, $\text{Loss}(S)$ is information loss measured against an oracle answer, and (α, β, γ) are deployment-specific weights.

B. Long-Context Baselines

Three families of strategies dominate the prior art.

Direct prompting. If $N \leq W$, the simplest strategy is $\mathcal{M}(q \parallel C)$. This is the gold standard when feasible but degrades sharply as N approaches W [12], [18]. On RULER [18], even frontier models that nominally support 128K tokens recall less than 50% of needles past 64K tokens.

Retrieval-augmented generation (RAG). A retriever R fetches top- k chunks from C relevant to q and the agent answers from $\mathcal{M}(q \parallel R(q, C))$ [19], [20]. RAG sharply reduces $\text{Tok}(S)$ but its recall ceiling is bounded by retriever quality, and it cannot answer queries that require aggregation across many chunks.

Summarization-and-compaction agents. Tools like Claude Code, Cursor, and OpenAI Codex periodically summarize the running context with a sub-LLM call, replacing the trajectory prefix with a summary [17]. This generalizes RAG to dynamic trajectories but suffers from compounding summarization error and the loss of verbatim quotes.

C. Recursive Language Models (RLM)

Zhang *et al.* [1] formalize an RLM as a tuple $(\mathcal{M}, \mathcal{E}, \rho)$ where $\mathcal{E}$ is a stateful environment (concretely, a Jupyter-style Python REPL) and ρ is a recursion depth bound. The full prompt C is bound to a global Python variable in $\mathcal{E}$ (e.g., `prompt = . . .`). The root LLM is given a system prompt that explains the affordances:

- 1) Read `len(prompt)` and `type(prompt)`.
- 2) Execute arbitrary Python (slicing, regex, NumPy, custom helpers).
- 3) Call `llm(sub_prompt, model=...)` to invoke a sub-LLM (depth 1) on a derived string. `Parallel llm_batch(prompts)` is supported.
- 4) Write the final answer to a designated `answer` variable and signal completion.

The sub-LLM at depth 1 is given only the slice it was called on and is, by default, not recursive (depth > 1 is supported but rarely required for current benchmarks [1]). The Prime Intellect `RLMEnv` additionally restricts tool access (web search, file I/O) to sub-LLMs, isolating tool-output bloat from the root context [2], [11].

Empirically, on OOLONG-Pairs at 128K input tokens, an RLM driven by GPT-5-mini achieves 51% accuracy while vanilla GPT-5 with the full prompt achieves 24%, at roughly half the cost per query [1]. On CodeQA and DeepResearch, similar 2–3 $\times$ improvements are reported.

D. Context Folding

Sun *et al.* [4] introduce two new actions into the agent’s action space: `branch(reason, plan)` and `return(summary)`. A `branch` pushes the current trajectory state onto an internal stack and starts a fresh sub-trajectory. A `return` pops the stack and replaces the entire branch with the supplied `summary` string in the parent’s context. The resulting trajectory is a *tree* of segments rather than a linear sequence; from the perspective of any single segment, only the segment’s own steps plus the parent’s pre-branch context plus prior siblings’ `summary`s are visible.

Sun *et al.* train this end-to-end via FoldPO, a PPO-variant with two process rewards: a *branch reward* encouraging the agent to branch when the predicted sub-trajectory will exceed a fold threshold, and a *summary reward* encouraging the summary to retain task-relevant information (measured by a downstream completion proxy). On Deep Research and SWE tasks, folding agents match the ReAct baseline at 10 $\times$ smaller active context and *outperform* agents constrained to the same active-context budget without folding [4].

E. Adjacent Lines of Work

Beyond RLM and folding proper, the broader landscape of *intra-task context curation* [5] includes: **MEM1** [38], which compresses the full history at each step; **MemAgent** [39], which uses a learned

memory module; **ReAcTree** [40], which decomposes goals into a dynamically constructed agent tree; **AgentFold** [5], which fine-tunes Qwen3-30B-A3B with a Fold-Generator data pipeline; and **AgentLoop / Reflexion** [22], which adds a verbal-reflection step on each iteration. We discuss the relative positioning of these in Section XI.

III. CONTEXT TOPOLOGY

The central conceptual move of HRCF is to make the agent’s working set explicit as a structured object that admits a per-fragment dispatch policy.

A. Definitions

A *context fragment* f is a contiguous span of text together with metadata. Formally,

$$f = \langle \text{id}, \text{tokens}, \text{provenance}, \text{access pattern} \rangle. \quad (2)$$

The access-pattern field is the key innovation. We define three values:

- **STATIC**, the fragment is read-only after instantiation. Examples: a user-uploaded PDF, a static codebase snapshot, a research corpus.
- **DYNAMIC**, the fragment is append-only and is the trajectory itself. The agent writes to it via tool calls, observations, and reasoning steps.
- **MIXED**, the fragment is read-only *and* re-read after intervening writes elsewhere. Examples: a slowly-evolving spec document; a long observation that the agent quotes verbatim multiple times.

The *context topology* $\mathcal{T}$ is a directed acyclic graph whose vertices are fragments and whose edges encode causal dependence: $f_i \rightarrow f_j$ if f_j is generated in the presence of f_i in context. The topology is constructed incrementally as the agent executes.

B. The Topology Classifier

For new fragments whose access pattern is not declared by the agent (in practice, this is the common case), HRCF runs a learned classifier $\phi : f \rightarrow \{\text{STATIC}, \text{DYNAMIC}, \text{MIXED}\}$. The classifier is a small (4-layer) MLP over hand-engineered features:

- Token length (log-transformed).
- Number of incoming write edges in $\mathcal{T}$ at fragment-creation time (always 0 at creation but updated retrospectively).
- Provenance type: file system, tool-call output, sub-LLM completion, or root-LLM completion.
- Syntactic-domain prior: code-token fraction, JSON-like structure indicator, prose readability score.
- Cosine similarity, in a frozen sentence-encoder embedding space, between the fragment and the original query.

We train ϕ on $\sim 8\text{K}$ manually-labelled fragments from a held-out set of public agent traces (collected from open-source LangChain, AutoGPT, and SWE-agent runs [26]). Test-set accuracy is 89.4%, with most errors confusing **STATIC** and **MIXED** on long observations that turn out to be quoted later. We find that simply biasing toward **MIXED** on borderline cases (a tunable threshold) is preferable to biasing toward **STATIC**, since the cost of a missed re-read is asymmetric (it forces a re-execution) relative to a spurious anchored-fold (which costs only a few hundred extra tokens).

C. An Illustrative Example

Consider a deep-research task: “Read these 47 academic PDFs and produce a comparative synthesis on context-management techniques in LLM agents.” At task start, the topology contains 48 fragments: the user query (MIXED, since it will be re-read on every reasoning step) and the 47 PDFs (STATIC). As the agent executes, it writes summaries (which themselves become fragments labelled MIXED if cross-referenced or DYNAMIC if append-only narrative), and it grows a synthesis draft (DYNAMIC). HRCF dispatches each fragment as it is read: the PDFs are all R-DELEGATED (each read is an RLM sub-call), the synthesis draft is F-BRANCHRETURN-folded once it exceeds a threshold, and the cross-referenced summaries are A-FOLD-folded with regex anchors so the final synthesis step can re-extract verbatim quotes when generating the comparison table.

IV. THE THREE PRIMITIVES

A. Primitive 1: R-DELEGATE

R-DELEGATE implements RLM-style sub-LLM dispatch over a Python-bound fragment. The fragment is bound to a uniquely-named REPL variable (`frag_f`), and the root LLM is given a system-prompt block describing the variable’s metadata. The root LLM may invoke a sub-LLM via `llm(sub_prompt, slice)` where `slice` is any Python expression evaluating to a string slice of `frag_f`.

The cost of R-DELEGATE on fragment f for a query that ultimately reads a set $S \subseteq f$ of token-ranges is

$$\text{Cost}_R(f, S) = \alpha(|S| + |q_{\text{sub}}| + |a_{\text{sub}}|) \cdot k + \beta \tau_{\text{rt}} k, \quad (3)$$

where $k = |S|/|s_{\text{max}}|$ is the number of sub-LLM calls (with $|s_{\text{max}}|$ the per-call slice cap), $|q_{\text{sub}}|$ and $|a_{\text{sub}}|$ are the average sub-prompt and sub-completion token counts, and τ_{rt} is the per-call round-trip latency. Information loss is zero if S is correctly chosen.

B. Primitive 2: F-BRANCHRETURN

F-BRANCHRETURN implements Sun-style branching. When the agent enters a sub-task whose expected length exceeds a threshold L_{fold} , it issues a `branch` action; on exit it issues `return(summary)`. The branch’s intermediate steps are removed from the parent context.

The cost of F-BRANCHRETURN on a branch that produces T trajectory tokens summarized to $T_s \ll T$ is

$$\text{Cost}_F(T, T_s) = \alpha(T + T_s) + \gamma \cdot \text{KL}(p_{\text{sum}} || p_{\text{full}}), \quad (4)$$

where the third term captures information loss as the KL divergence between the conditional answer distribution under the summary versus the full branch (estimated by a held-out LLM judge).

C. Primitive 3: A-FOLD (Anchored Fold)

A-FOLD is a refinement of F-BRANCHRETURN for fragments labelled MIXED. In addition to a natural-language summary σ , the agent emits a *handle table* H of $\langle p_i, [b_i^{\text{lo}}, b_i^{\text{hi}}], \ell_i \rangle$ triples, where p_i is a regex pattern, $[b_i^{\text{lo}}, b_i^{\text{hi}}]$ is the byte-offset range in the original branch trajectory, and ℓ_i is a semantic label (e.g., “error message from line 47” or “user requirement #3”). The original branch trajectory is preserved on disk but evicted from the LLM context.

When a later step needs a verbatim quote, it issues `anchor_lookup(label)` or `anchor_grep(pattern)`. The lookup is a constant-cost disk read; no LLM call is required.

If the lookup misses (the desired content is not anchored), the step falls back to re-execution at full F-BRANCHRETURN cost.

The anchored-fold cost is

$$\text{Cost}_A(T, T_s, H) = \alpha(T + T_s + |H|) + \alpha \cdot \rho \cdot |H_{\text{hit}}| + \alpha \cdot \rho_{\text{miss}} \cdot \text{Cost}_F(T, T_s) + \gamma \cdot \text{KL}(p_{\sigma+H} \| p_{\text{full}}), \quad (5)$$

where ρ is the empirical re-reference rate (fraction of later steps that quote the branch), $|H_{\text{hit}}|$ is the average size of an anchor hit, and ρ_{miss} is the miss rate. Importantly, the KL term is strictly smaller than for F-BRANCHRETURN because the handle table preserves verbatim spans for the fraction $1 - \rho_{\text{miss}}$ of later quotes.

D. The Dispatch Policy

Given a fragment f with access pattern $\pi(f)$ and predicted re-reference rate $\hat{\rho}(f)$ (estimated by the same classifier ϕ), HRCF dispatches as follows:

$$\text{dispatch}(f) = \begin{cases} \text{R-DELEGATE} & \text{if } \pi(f) = \text{STATIC} \\ \text{F-BRANCHRETURN} & \text{if } \pi(f) = \text{DYNAMIC} \wedge \hat{\rho}(f) < \rho_0 \\ \text{A-FOLD} & \text{if } \pi(f) = \text{MIXED} \vee \hat{\rho}(f) \geq \rho_0 \end{cases} \quad (6)$$

where $\rho_0 \in [0, 1]$ is a tunable threshold (we use $\rho_0 = 0.4$ in all experiments; see Section X).

Algorithm 1 HRCF dispatch and execution loop

Require: Query q , base model $\mathcal{M}$, classifier ϕ , REPL env $\mathcal{E}$

```

1: Initialize topology  $\mathcal{T} \leftarrow \emptyset$ , working context  $W \leftarrow [q]$ 
2: while not terminated do
3:   Sample action  $a \sim \mathcal{M}(\cdot \mid W)$ 
4:   if  $a$  creates a new fragment  $f$  then
5:      $\pi(f), \hat{\rho}(f) \leftarrow \phi(f)$ 
6:      $\mathcal{T} \leftarrow \mathcal{T} \cup \{f\}$  with edges from  $W$ 
7:      $\text{prim}(f) \leftarrow \text{dispatch}(f)$  via Eq. 6
8:     Bind  $f$  in  $\mathcal{E}$  if  $\text{prim}(f) = \text{R-DELEGATE}$ 
9:   end if
10:  if  $a$  is a branch action then
11:    Push current  $W$ , start fresh sub-context
12:  else if  $a$  is a return( $\sigma$ ,  $H$ ) action then
13:    Pop sub-context, append  $\sigma$  to parent  $W$ 
14:    Persist  $H$  to anchor store if non-empty
15:  else if  $a$  is an anchor_lookup then
16:    Append disk-read result to  $W$ 
17:  else
18:    Update  $W$  with  $a$ 's effect
19:  end if
20: end while
21: return final answer in  $W$ 

```

V. THEORETICAL ANALYSIS

We prove a competitive-ratio bound for HRCF’s dispatch policy against the offline per-fragment optimum.

A. Setting

Let $\mathcal{F} = \{f_1, \dots, f_n\}$ be the set of fragments encountered during a task, and let σ be the (unknown to the online algorithm) sequence of accesses to fragments. The *offline optimum* OPT is the cost of the dispatch that, knowing σ in advance, assigns each fragment to the primitive that minimizes its total cost. The *online algorithm* (HRCF) sees fragments one at a time and must commit to a primitive at fragment-creation time.

B. Theorem: Competitive Ratio

Theorem 1 (Competitive Ratio of HRCF Dispatch): Under the assumptions that (i) the cost-model parameters (α, β, γ) are bounded and known, (ii) the topology classifier ϕ has bounded calibration error ϵ_ϕ , and (iii) the maximum recursion depth is d , HRCF’s dispatch achieves

$$\mathbb{E}[\text{Cost}(\text{HRCF})] \leq (2 + \log_2 d + 2\epsilon_\phi) \cdot \text{OPT}. \quad (7)$$

Proof Sketch. The proof proceeds in three steps.

Step 1: Reduction to weighted-caching. We model the choice between R-DELEGATE and A-FOLD as a caching problem: each fragment’s verbatim content is either “in cache” (kept addressable via REPL variable or anchor table) or “evicted” (only summary remains). The cost of an access on an evicted fragment is the re-execution cost; the cost of an access on a cached fragment is the access cost plus the per-step cache rent. Standard results [23], [24] give an $O(\log k)$ -competitive ratio for k -cache, where k is the cache size; with cache size d (the recursion depth), this yields the $\log_2 d$ term.

Step 2: Classifier calibration. The classifier’s bounded calibration error ϵ_ϕ contributes additively because the dispatch decision (Eq. 6) is correct *a fortiori* when the classifier is correct, and incorrect by at most a factor of $1 + \epsilon_\phi$ on misclassified fragments (since the worst alternative primitive is bounded by a constant times the optimal one).

Step 3: F-BRANCHRETURN is constant-competitive. For DYNAMIC fragments, F-BRANCHRETURN is at most 2-competitive against the trivial-summary baseline (within a constant factor of the offline summary-length-aware optimum [25]).

Combining the three steps, the total competitive ratio is $2 + \log_2 d + 2\epsilon_\phi$ as claimed. $\square$

C. Discussion of the Bound

For the depths we observe in practice ($d \leq 4$ in 99% of tasks), $\log_2 d \leq 2$, and our trained classifier achieves $\epsilon_\phi \approx 0.06$, yielding an empirical competitive ratio of approximately 4.1. This is loose; in practice we observe HRCF within $1.4\times$ of the offline optimum on synthetic tasks where we can compute OPT exactly; but the bound is non-vacuous and, importantly, depth-dependent in a way that motivates Section X’s recommendation to cap recursion at $d = 3$ in production.

D. Lower Bound

A matching lower bound is straightforward: any deterministic online dispatch that does not see future accesses is at least $\log_2 d$ -competitive on the adversarial weighted-caching instance from [23]. We omit the details.

VI. THE ANCHORED-FOLD PRIMITIVE IN DEPTH

The novelty of A-FOLD relative to prior context-folding work is the explicit handle table. In this section we detail its construction and use.

A. Handle Generation

When the agent issues `return`, HRCF prompts the agent with a structured sub-prompt:

You are about to return from this branch. The trajectory contained $\{T\}$ tokens. Produce: (1) a concise summary σ ($\leq T_s$ tokens) capturing the task-relevant outcome; (2) a handle table of up to 16 anchor entries, each a JSON object with `'pattern'` (a regex), `'label'` (a short semantic name), and `'priority'` (1-5). Anchor entries should target spans that downstream steps may need to quote verbatim.

The handle entries are validated: each pattern is compiled and matched against the original branch trajectory; matches that overlap or that exceed 1024 tokens are pruned. The validated table is persisted to an on-disk key-value store keyed by branch ID.

B. Anchor Lookup Semantics

A subsequent step may issue:

- `anchor_lookup(label)`: returns the verbatim text of the highest-priority entry whose label string-matches.
- `anchor_grep(pattern)`: returns up to 5 verbatim matches of the pattern across all anchored branches in the current ancestry.
- `anchor_list()`: returns the labels and priorities of all anchored entries, useful for the agent to discover what is available.

All three are constant-cost (disk reads, no LLM call). On a miss, the agent may issue `branch_replay(branch_id)` which restages the original branch as a fresh sub-trajectory; this is the fallback that recovers correctness at the cost of recomputation.

C. Empirical Behaviour of Anchors

On a synthetic mixed-access benchmark (Section VIII), we observe that handles produced by GPT-4o-mini-driven agents have a 73% *hit rate*, the fraction of later quote-attempts satisfied by an anchor lookup. The remaining 27% split into 19% correctly-resolved by `anchor_grep` and 8% requiring full `branch_replay`. The total-cost benefit relative to pure F-BRANCHRETURN is 24% token reduction and 11% accuracy improvement on this benchmark (Section IX).

VII. REFERENCE IMPLEMENTATION

We release HRCF as a 1,400-line Python library with three integration points.

A. Architecture

The implementation is organized as follows:

- `hrcf/topology.py` (220 lines): the topology graph data structure, fragment objects, and the classifier ϕ (PyTorch).
- `hrcf/primitives.py` (380 lines): the three primitives, including the REPL wrapper for R-DELEGATE (extends the `rlm` package [10]) and the branch/return state machine for F-BRANCHRETURN (extends the Context-Folding reference release [4]).
- `hrcf/anchors.py` (170 lines): the on-disk key-value anchor store, regex compilation, and lookup APIs.
- `hrcf/dispatcher.py` (150 lines): the online dispatch policy, cost-model implementation, and the main agent loop (Algorithm 1).
- `hrcf/backends/` (380 lines across multiple files): adapters for the OpenAI API, Anthropic API, and local OpenRouter-compatible endpoints.
- `hrcf/eval/` (100 lines): benchmark harnesses for MD-NIAH, OOLONG-Pairs, and the SWE simulation.

B. Key Design Decisions

REPL isolation. R-DELEGATE runs in either a `LocalREPL` (in-process; fast but unsafe for production) or a `DockerREPL` (sandboxed Docker container; slower but safe for production deployment), following the architecture of [10].

Anchor store backend. Anchor tables are stored in a SQLite database in the working directory, keyed by branch ID and label. We considered Redis but SQLite proved sufficient at our scales and avoids external dependencies.

Streaming and caching. Sub-LLM completions are streamed to allow early termination; the implementation supports response caching across runs to enable cheap reproduction.

VIII. EXPERIMENTAL SETUP

A. Benchmarks

We evaluate on three benchmarks chosen to span the access-pattern space.

(B1) Multi-Document Needle-in-a-Haystack (MD-NIAH). A synthetic benchmark we construct as follows: given K documents (each a passage from the Pile [27]), M *needles* (factoids of the form “the secret word in document i is w_i ”) are inserted at random positions; the query is to enumerate the secret words in document-id order. We vary $K \in \{8, 32, 128, 512\}$ and pad to total token budgets of $\{32K, 128K, 512K, 2M\}$. This is the prototypical STATIC-fragment benchmark.

(B2) OOLONG-Pairs. We use the public re-implementation of OOLONG-Pairs [7]: given a long context of unstructured prose, the model must find all pairs of entities satisfying a relation (e.g., “persons who lived in the same city”). This is the canonical aggregation-over-static-context benchmark and was the headline benchmark in the RLM paper [1]. We evaluate at the same input lengths Zhang *et al.* report.

(B3) SWE-Sim. We adapt the long-horizon software-engineering simulation harness from the public Context-Folding release. Each task asks the agent to make a multi-file change in a synthetic codebase (10–40 files), test it, and produce a patch. This is a DYNAMIC-trajectory benchmark with extensive verbatim quoting of error messages and file contents; the prototypical MIXED regime.

B. Models

We use three base models to span the open/closed and small/large axes:

- **GPT-4o-mini** (closed, small): the primary system used for ablations because of cost. All ablations and depth experiments are run with GPT-4o-mini.
- **GPT-4o** (closed, large): used for the headline numbers in Section IX.
- **Qwen3-8B-Instruct** (open, small): used to confirm that HRCF gains generalize beyond closed proprietary models [29].

C. Baselines

We compare HRCF against five baselines:

- 1) **Direct** (the model is given the full prompt up to its context window; truncated otherwise).
- 2) **RAG-BM25** (top-32 chunks retrieved by BM25, then direct prompt).
- 3) **RAG-Embedding** (top-32 chunks retrieved by `text-embedding-3-large` [28]).
- 4) **Pure-RLM** (the open-source `rlm` package [10] with depth $d = 1$).
- 5) **Pure-Folding** (the public Context-Folding agent [4] with default thresholds).

D. Metrics

We report:

- **Accuracy** (task-specific: needle-recall for MD-NIAH, F1 for OOLONG-Pairs, test-pass rate for SWE-Sim).
- **Total tokens** (input + output) summed over all model calls.
- **Wall-clock latency** (median over 5 runs, single-tenant).
- **Cost per query** (USD, computed from public API pricing).

E. Reproducibility

All API calls are seeded where the API supports it; for non-deterministic backends (Anthropic, OpenRouter open models) we report $\text{mean} \pm \text{std}$ over 5 runs. Code, prompts, and anchored-store dumps are released at the project repository.

IX. RESULTS

A. Headline Numbers

Table I summarizes the headline results across all three benchmarks with GPT-4o as the base model. HRCF matches or exceeds the better of Pure-RLM and Pure-Folding on every benchmark, with consistent reductions in token cost and latency.

B. Scaling with Input Length

On MD-NIAH, accuracy as the input length grows from 32K to 2M tokens shows the following pattern. Direct is at-capacity at 32K; both RAG methods plateau by 128K; Pure-RLM and Pure-Folding scale gracefully but diverge from each other past 512K (Pure-RLM has the edge as input grows because folding’s information loss compounds across many summarization steps). HRCF tracks the upper envelope and at 2M tokens matches Pure-RLM accuracy (69.4 vs 68.7%) at 34% fewer tokens.

TABLE I
HEADLINE RESULTS: GPT-4O, ALL THREE BENCHMARKS. BOLD INDICATES BEST. HRCF MATCHES OR EXCEEDS THE STRONGER OF PURE-RLM/PURE-FOLDING ON ACCURACY WHILE REDUCING TOKENS AND LATENCY.

Method	Acc. (%)	Tokens (K)	Latency (s)	Cost (\$)
<i>B1: MD-NIAH at 512K input</i>				
Direct (truncated)	22.1	128.0	12.3	0.640
RAG-BM25	41.7	18.4	4.1	0.092
RAG-Embedding	56.2	18.6	4.4	0.093
Pure-RLM	79.3	47.2	31.4	0.236
Pure-Folding	71.8	38.5	22.7	0.193
HRCF (ours)	84.1	29.3	14.6	0.147
<i>B2: OOLONG-Pairs at 128K input</i>				
Direct	27.4	128.0	9.8	0.640
RAG-BM25	31.2	14.7	3.6	0.074
RAG-Embedding	38.9	14.9	3.9	0.075
Pure-RLM	53.1	41.6	28.2	0.208
Pure-Folding	47.6	35.1	19.5	0.176
HRCF (ours)	58.4	27.8	13.7	0.139
<i>B3: SWE-Sim, 30-file tasks</i>				
Direct	18.2	96.0	22.1	0.480
RAG-BM25	24.6	31.2	9.7	0.156
RAG-Embedding	28.3	31.5	10.1	0.158
Pure-RLM	41.7	88.4	56.9	0.442
Pure-Folding	51.2	52.3	32.6	0.262
HRCF (ours)	55.8	38.1	27.4	0.191

C. Generalization Across Models

Table II reports HRCF’s results with all three base models. The relative gain over the strongest baseline is preserved: +4.8 pp on GPT-4o-mini, +4.6 pp on GPT-4o, and +5.9 pp on Qwen3-8B-Instruct (averaged over benchmarks). The Qwen3-8B-Instruct gain is the largest, consistent with Zhang *et al.*’s finding [1] that smaller models benefit more from delegated context management.

TABLE II
HRCF GAIN OVER STRONGEST SINGLE-METHOD BASELINE, AVERAGED ACROSS B1–B3.

Model	Avg. accuracy gain (pp)	Avg. token reduction (%)
GPT-4o-mini	+4.8	31.2
GPT-4o	+4.6	33.7
Qwen3-8B-Instr.	+5.9	27.4

D. Cost-Quality Frontier

A central claim of this work is that HRCF dominates a Pareto frontier in $\langle \text{accuracy}, \text{token-cost} \rangle$ space. We confirm this by sweeping each method’s main hyperparameter (number of retrieved chunks for RAG, recursion depth for Pure-RLM, fold threshold for Pure-Folding, and dispatch threshold ρ_0 for HRCF) and plotting the resulting frontier on B1. HRCF’s frontier strictly dominates Pure-RLM’s at all token budgets above 20K, and dominates Pure-Folding’s at all token budgets above 25K. Below 20K the dominance flips: pure-RAG and Direct (when feasible) are cheaper.

X. ABLATIONS

A. Component Ablation

We disable each HRCF component in isolation, holding all else constant on B2 (OOLONG-Pairs at 128K). Table III reports the absolute accuracy drop.

TABLE III
ABLATION ON B2. REMOVING EACH COMPONENT IN ISOLATION; ACCURACY DROP IS IN PERCENTAGE POINTS.

Configuration	Δ Accuracy (pp)
Full HRCF (baseline)	0.0
– Topology classifier (ϕ)	−11.4
– Anchored fold (A-FOLD $\rightarrow$ F-BR)	−4.7
– Per-fragment cost model (uniform ρ_0)	−3.1
– Both anchored and cost model	−7.9
– All three (degrades to alternating R/F)	−12.8

B. Sensitivity to ρ_0

We sweep $\rho_0 \in [0.1, 0.9]$. Performance is stable in $[0.3, 0.5]$, which we take as the recommended range. At $\rho_0 = 0$ the system collapses to Pure-Folding (−7.5 pp); at $\rho_0 = 1$ it collapses to Pure-RLM (−3.2 pp).

C. Recursion Depth

We vary the maximum recursion depth $d \in \{1, 2, 3, 4\}$. Performance peaks at $d = 2$; $d = 3$ yields a 0.3-pp gain at $2.1\times$ token cost; $d = 4$ degrades performance, consistent with prior reports of diminishing returns past depth 1 in pure RLM [1]. We recommend $d = 2$ in production.

D. Topology Classifier Variants

We replace the trained MLP with: (a) a length-only heuristic (STATIC if $|f| > 8K$, else DYNAMIC), (b) a zero-shot LLM classifier (GPT-4o-mini prompted with the labelling rubric), and (c) an oracle classifier with ground-truth access labels.

The trained MLP is within 1.4 pp of the oracle and 5.8 pp ahead of the length-only heuristic. The zero-shot LLM classifier is competitive (−1.9 pp vs. trained) but $\sim 14\times$ more expensive per-classification; we recommend the trained MLP for cost reasons but note the zero-shot variant is a reasonable drop-in for low-traffic deployments.

E. Negative Results

We document two regimes where HRCF *underperforms*:

- 1) **Short inputs** ($N \leq 32K$). HRCF’s overhead (classifier inference, REPL setup, anchor-store writes) is not amortized; Direct prompting wins by 2–3 pp at lower latency. Our recommendation: gate HRCF behind a length threshold of 32K tokens.
- 2) **Highly homogeneous tasks**. On a synthetic “find all sentences containing the word ‘red’ ” task, where every fragment is touched in the same way, the topology classifier’s three-way dispatch is wasted; Pure-RLM is 1.1 pp better. The dispatch overhead matters relative to the size of the work being dispatched.

These negative results are not surprising, HRCF’s value proposition is in heterogeneity, and are useful for practitioners deciding when to deploy.

XI. RELATED WORK

A. Recursive Language Models and Variants

Beyond the canonical Zhang [1] and Prime Intellect [2] treatments, several concurrent works explore recursive scaffolds. Schroeder *et al.* [30] study task-decomposition recursion (without context delegation), and Sun *et al.* [31] study recursive sampling. The closest pre-RLM work is DSPy’s Module-as-callable abstraction [21], which permits sub-LLM calls but not over arbitrary slices of a Python-bound prompt. The *Hierarchical Reasoning Model (HRM)* [32] provides a complementary depth dimension by training models to recursively re-encode hidden states; HRCF and HRM are largely orthogonal.

B. Context Engineering

Mei *et al.* [33] survey context engineering as an emerging research area; Qiao *et al.* [34] provide a taxonomy. External context augmentation [35]–[37] retrieves from outside the current trajectory; intra-task curation is the family our work belongs to. MEM1 [38] and MemAgent [39] compress at every step (no anchoring); AgentFold [5] adds proactive folding via SFT; FoldAct [6] stabilizes self-conditioning during folded rollouts via a consistency loss. None separates static from dynamic fragments, and none offers anchor-table retrieval into folded summaries.

C. Inference-Time Scaling

Snell *et al.* [8] formalize the test-time-vs-training-time compute tradeoff; OpenAI’s o1 and o3 [41], DeepSeek R1 [42], and the budget-forcing approach of Muennighoff *et al.* [43] all demonstrate that more inference compute yields better answers. HRCF is in some sense an inference-time-compute strategy specialized for the long-context regime: it spends more compute (multiple sub-LLM calls, anchor lookups) to retain accuracy as context grows. Our paper II in this submission explores the inference-scaling angle directly.

D. Recursive Self-Improvement

The ICLR 2026 RSI Workshop [9] has crystallized RSI as an engineering discipline, with 110 accepted papers spanning self-play, automated AI research, and self-evolving agents. HRCF is not an RSI system per se, it does not modify its own weights or prompts, but the FoldPO direction [4] of training a model natively on the HRCF scaffold is an instance of the experience-learning lens in the RSI taxonomy.

E. Multi-Agent and Tool-Use Systems

Anthropic’s multi-agent research system [17] uses a coordinator-subagent split that is structurally analogous to RLM’s root-sub LLM split, but at coarser granularity (sub-agents are independent processes with separate APIs). MetaGPT [44] and Magentic-One [45] are multi-agent frameworks; their orthogonal contribution is role specialization. We see HRCF as composable with these systems: each agent in a multi-agent system could internally use HRCF for its own context management.

F. Memory Architectures

A separate line of work attempts to give LLMs explicit long-term memory. Park *et al.* [68] introduced generative-agent memory streams; MemGPT [69] treats the LLM as an OS-style process with virtual memory; Mem0 [70] offers a production memory framework. These systems target inter-task continuity and personalization, complementary to HRCF’s intra-task focus. A unified memory-plus-HRCF architecture is a natural future direction.

G. Compression-Based Long-Context

A separate family attempts to fit long contexts into shorter ones via learned compression. AutoCompressors [71] use auxiliary tokens that distill a long history; LLMingua [72] prunes tokens via a small importance model; ICAE [73] learns a soft prefix that encodes the long context. Compression is orthogonal to HRCF: a compressed prefix can still be wrapped in HRCF’s dispatch policy as a STATIC fragment.

H. Position Relative to Standardization

The Model Context Protocol (MCP) [74] and Agent-to-Agent (A2A) [75] efforts standardize *external* agent interfaces (tool calls, agent-to-agent communication). HRCF operates one layer down: it concerns the *internal* context of a single agent. The two are composable: an MCP tool call’s response is a fragment that HRCF would label and dispatch like any other.

XII. NATIVE TRAINING OF HRCF POLICIES

A central observation in the RLM [1] and FoldPO [4] literatures is that an inference-time scaffold can be substantially improved by training the underlying base model natively to operate within it. Zhang *et al.* report a 28.3% average gain for RLM-Qwen3-8B over base Qwen3-8B; Sun *et al.* report similar magnitudes for FoldPO-trained agents. We initiate this direction for HRCF.

A. Training Setup

We collect 200 HRCF trajectories on B1, B2, and B3 using GPT-4o as a teacher (running standard inference-time HRCF). Each trajectory is logged in a structured format that records, for every action, the fragment the action targets, the chosen primitive, the resulting fragment, and the eventual task reward. We then perform supervised fine-tuning on Qwen3-8B-Base using the trajectories as training data, with a loss that masks out tokens generated by sub-LLMs (we want to train only the root LLM’s policy).

B. Preliminary Results

The SFT-trained model, which we call HRCF-Qwen3-8B, outperforms the inference-time-only HRCF on B2 by 4.2 pp absolute (61.7% vs. 57.5% with the same Qwen3-8B base). Token usage drops by an additional 11% relative to inference-only HRCF, owing to fewer redundant sub-LLM calls (the trained model learns to amortize work across calls more efficiently).

These numbers are preliminary; we have not yet run a full RL phase analogous to FoldPO, and the SFT is small (200 trajectories versus the thousands typically used for FoldPO). We expect substantial additional gains from a proper RL programme. We release the SFT recipe and weights as supplementary material.

C. Reward Design Considerations for Future RL

For a future RL phase, we recommend:

- **Process reward for primitive choice:** a small positive reward when the chosen primitive matches the offline-optimal primitive (computed retrospectively from full trajectory).
- **Anchor-utility reward:** a positive reward proportional to the fraction of emitted anchors that are subsequently used (with a small penalty for unused anchors).
- **Token-efficiency reward:** negative reward proportional to token cost above a baseline, encouraging the model to be parsimonious.
- **Final-task reward:** the dominant signal, identical to FoldPO [4].

A weighted combination of these, tuned via population-based training, is the natural starting point.

XIII. PRACTICAL DEPLOYMENT CONSIDERATIONS

We summarize practical guidance derived from our experiments and pilot deployments.

A. When to Deploy HRCF

Use HRCF when total input or trajectory length is expected to exceed 32K tokens, when tasks are heterogeneous in access pattern (a mix of static document inputs and dynamic agent state), when production budget is sensitive to per-query token cost, or when tasks involve verbatim quoting from prior content. Use simpler scaffolds when inputs are short and homogeneous, when latency is critical, or when tasks are entirely static or entirely trajectory-based.

B. Tuning ρ_0 in Production

We recommend starting with $\rho_0 = 0.4$ and adjusting based on the observed mix of fragments. If anchor lookups have a low hit rate ($< 50\%$), the threshold is too aggressive (favouring A-FOLD too often); raise to 0.5. If sub-LLM call cost dominates, the threshold is too conservative; lower to 0.3. Periodic re-tuning every $\sim 10,000$ trajectories is sufficient in our experience.

C. Logging and Debugging

The HRCFLogger captures the entire dispatch decision tree. The visualizer (extending rlm’s visualizer [10]) renders the topology DAG with node colour indicating primitive choice and edge weight indicating cross-fragment access. We have found this visualizer essential for diagnosing premature-folding bugs in production.

D. Safety in Production

Three concrete recommendations: (i) run the REPL in a Docker container with no network access and a 30-second wall-clock cap per cell; (ii) sandbox sub-LLM calls behind a per-task budget (e.g., max 20 sub-calls per task); (iii) audit anchor-store contents periodically, since anchored verbatim content can include sensitive information that survived a fold and should be subject to the same retention policies as the underlying inputs.

XIV. DISCUSSION

A. When HRCF Helps and When It Doesn't

The central empirical message is that hybridization helps when fragments differ in access pattern, and hurts when they do not. This matches intuition: a homogeneous workload need not pay the dispatch overhead. We have found in pilot deployments that real-world agentic tasks, deep research, software engineering, document review, are heterogeneous enough that HRCF's overhead is amortized; pure-synthetic and pure-RAG-style tasks are more homogeneous and may not benefit.

B. Training-Time Implications

Zhang *et al.* [1] train RLM-Qwen3-8B natively on the RLM scaffold and observe a 28.3% average gain. Sun *et al.* [4] train via FoldPO and observe similar gains. Both findings suggest the next research step for HRCF: train a native HRCF policy. We have begun this work with a 200-trajectory SFT on Qwen3-8B-Base; preliminary results show a 4.2-pp accuracy gain on B2 over the inference-time-only HRCF, but proper hyperparameter search and longer RL training are future work.

C. Safety Considerations

The R-DELEGATE primitive permits arbitrary Python execution. Production deployments should use the `DockerREPL` backend with restricted system calls, no network access for the root REPL (sub-LLMs may have controlled tool access), and a per-task wall-clock cap. We note that the same considerations apply to vanilla RLM [2] and refer the reader to those discussions for deployment best practices.

D. Limitations

- 1) HRCF is an inference-time framework; without native training, the underlying model's policy may be sub-optimal for the dispatch decisions HRCF makes (e.g., a model might continue generating in a fragment that should have been folded).
- 2) The topology classifier is trained on a specific distribution of agent traces; deployment in a domain with very different fragment statistics may require re-training. We provide a fine-tuning recipe in the supplementary appendix.
- 3) The competitive-ratio bound in Theorem 1 assumes bounded cost-model parameters; in practice (α, β, γ) may shift with API pricing changes, requiring re-tuning of ρ_0 .
- 4) We do not evaluate on multimodal long contexts (video, audio); this is significant future work given the rise of multimodal agents.

E. Open Problems

Three problems we did not solve:

- 1) **Dynamic anchoring.** Currently anchors are emitted only at `return` time. Anchoring *during* a branch (predicting which spans will be referenced later) is a natural next step.
- 2) **Cross-task anchor sharing.** Anchors are scoped to a single task; reusing anchors across tasks (a form of long-term memory) raises interesting privacy and staleness questions.
- 3) **Deeper recursion.** We saw diminishing returns past $d = 2$, but our experiments do not stress benchmarks where deep recursion is fundamental (e.g., proof search). A study of when recursion depth matters is open.

XV. EXTENDED EMPIRICAL STUDIES

In this section we report extended studies that did not fit in the headline results section: cross-domain transfer, robustness to adversarial fragments, the role of model scale, and a qualitative analysis of HRCF trajectories.

A. Cross-Domain Transfer

The topology classifier ϕ is trained on a mixture of LangChain, AutoGPT, and SWE-agent traces. We test its transfer to two out-of-distribution domains: legal document review (drawn from a public corpus of 1,200 contract-analysis traces released by [66]) and biomedical literature synthesis (from PubMedQA-style multi-paper queries). On legal review, classifier accuracy drops from 89.4% in-domain to 81.7%, and HRCF’s benefit over Pure-Folding shrinks from +5.4 pp to +2.9 pp. On biomedical, accuracy is 84.0% and the benefit is +4.1 pp. The benefit is preserved but smaller, and is recovered to within 1 pp of in-domain after a 200-trace fine-tune of ϕ on each domain. We provide the fine-tuning script in the supplementary material.

B. Robustness to Adversarial Fragments

We construct an adversarial test in which fragments are crafted to mislead the topology classifier: short fragments that are *actually* re-referenced many times (mislabelling them as DYNAMIC would cost recall), and very long fragments that are *actually* read once (mislabelling them as STATIC would waste tokens on REPL setup). Performance degrades by 3.2 pp on the short-but-mixed adversarial set and 1.8 pp on the long-but-static adversarial set. The asymmetry is consistent with the asymmetric classifier-bias recommendation in Section III (bias toward MIXED).

C. Effect of Model Scale

Within the GPT-4 family, we compare GPT-4o-mini (small), GPT-4o (medium), and GPT-4.1 (large; via the OpenAI API as of February 2026). Pure-RLM gains shrink with scale (small models benefit more from delegation [1]), but HRCF’s marginal gain over Pure-RLM is roughly constant: +4.8 pp small, +4.6 pp medium, +4.1 pp large. We interpret this as evidence that the gains are dominated by access-pattern dispatch, not by the underlying delegation, and so do not vanish as the base model improves.

D. Trajectory Walk-Through: A Worked Example

To make HRCF’s behaviour concrete, we walk through a single trajectory on a simplified deep-research task: “Read these three abstracts; identify the methodological similarity that is shared by exactly two of them; produce a 100-word synthesis.” The three abstracts are about, respectively, RLMs [1], context folding [4], and an unrelated work on attention pruning [53]. The query forces the model to read all three, then aggregate.

a) *Step 0: Topology construction:* HRCF receives the three abstracts and the query. The classifier ϕ labels the abstracts STATIC (high probability 0.87, 0.91, 0.83); the query is labelled MIXED (it will be re-read on every reasoning step). Each abstract is bound to a REPL variable.

b) *Step 1: Read:* The root LLM emits Python code:

```
1 abs1_summary = llm("Summarize methodology", frag_a1)
2 abs2_summary = llm("Summarize methodology", frag_a2)
3 abs3_summary = llm("Summarize methodology", frag_a3)
```

Three sub-LLM calls fire in parallel (`llm_batch`). The summaries are bound to root-LLM-visible variables.

c) *Step 2: Compare:* The root LLM emits:

```
1 common = llm("Identify shared methodology among ANY 2:",
2             [abs1_summary, abs2_summary, abs3_summary])
```

A single sub-LLM call returns: “Both RLMs and context folding decompose long inputs/trajectories into smaller units processed independently; the third does not.”

d) *Step 3: Synthesize:* The root LLM emits a final `answer-binding` statement that uses `common` and the three summaries. No anchors are needed (the abstracts are short and the summaries are sufficient). Total: 3 depth-1 sub-LLM calls plus root reasoning. Pure-RLM would have made the same number of calls; Pure-Folding would have read all three abstracts in the root context and required no folding (input fits). HRCF’s overhead here is modest, the topology classifier runs three times, but on this task no method has a meaningful advantage. The point of the walk-through is to show that HRCF’s behaviour reduces to Pure-RLM in the all-STATIC regime, by design.

A more interesting trajectory occurs when one of the abstracts grows to 50K tokens (a survey paper instead of an abstract). The classifier now labels that fragment STATIC but flags its high length, triggering chunked sub-LLM dispatch. The other two abstracts are still read in their entirety. The root reasoning is unchanged.

E. Failure Modes

We catalogue the three most common failure modes observed in $\sim 2,000$ trajectories:

- 1) **Anchor over-emission.** The agent emits more anchor entries than necessary, increasing token cost without corresponding accuracy benefit. Mitigation: cap handle table at 16 entries (we did so); penalize unused anchors in any future native training.
- 2) **Premature folding.** On dynamic trajectories, the agent occasionally folds before the sub-task is complete, losing necessary detail. The summary then fails to capture an outcome that was about to be produced. Mitigation: a learned “fold readiness” signal (future work) or a hand-tuned minimum branch length (current).
- 3) **Sub-LLM hallucination on slices.** A depth-1 sub-LLM occasionally invents content that is not present in the slice it was given. This is a known failure of all RLM-style systems [1], [2] and is mitigated only by stronger base models or verifier-guided sampling. We did not innovate here.

F. Latency Profiling

To understand HRCF’s wall-clock characteristics we instrumented the system and report cumulative latency by phase across 500 B1 trajectories at 512K input length. Sub-LLM round-trips dominate (mean 74% of total time), root reasoning is second (16%), classifier inference and dispatch overhead are negligible (1.4%), and anchor I/O is 2.1%. Network latency dominates the round-trip cost; in a local-cluster deployment with a co-located LLM server, total latency drops by $\sim 40\%$ and the relative cost of dispatch overhead grows correspondingly. This suggests that HRCF benefits disproportionately from infrastructure that minimizes round-trip latency, a useful pointer for production deployment.

G. Energy and Carbon Considerations

Test-time compute increases per-query energy use; this is a known concern for the inference-scaling paradigm broadly [46], [67]. We estimate per-query energy from public API token-throughput numbers: HRCF on B2 consumes roughly 32% less energy than Pure-RLM and 20% less than Pure-Folding, owing to its lower total token count. Energy efficiency is therefore an additional axis on which HRCF dominates among scaffolds at the same accuracy level, though all scaffolds are more energy-intensive than direct prompting (which is, however, infeasible past the context window).

XVI. CONCLUSION

We have presented HRCF, a hybrid framework that unifies recursive language models and context folding under a single per-fragment dispatch policy. The framework rests on three primitives, R-DELEGATE, F-BRANCHRETURN, and the novel A-FOLD, a topology formalism that classifies fragments by access pattern, and a cost model under which the dispatch is provably $O(\log d)$ -competitive against the offline optimum. Empirically, HRCF matches or exceeds the better of pure-RLM and pure-folding across three benchmarks at 27–41% lower token cost. The largest-impact component is the topology classifier; the next-largest is the anchored-fold primitive.

We see HRCF as a step toward a unified theory of context engineering. As inference-time compute continues to grow as a fraction of total AI economic activity [46], the question of how to spend that compute well, in particular, how to spend it on context management rather than naive recall, will only become more central. HRCF demonstrates that progress here is not about choosing one paradigm over another; it is about building the meta-policy that knows which to use when.

ACKNOWLEDGMENTS

The author thanks the open-source maintainers of the `rlm` [10] and Context-Folding reference codebases, whose releases made this work possible, and the ICLR 2026 RSI Workshop organizers for an open call that helped frame the broader research programme.

APPENDIX A PROOF OF THEOREM 1

We give a complete proof of the competitive ratio claimed in Section V.

A. Setup and Notation

Let $\sigma = (a_1, a_2, \dots, a_T)$ denote the sequence of fragment accesses generated during a task, where each $a_t \in \mathcal{F}$ identifies the fragment accessed at step t . Let $\text{prim}_t(f) \in \{R, F, A\}$ denote the primitive used by the algorithm to handle fragment f at the time of access t (the primitive may have been chosen at fragment creation but is "in effect" at t).

The cost of an algorithm $\mathcal{A}$ on instance $(\sigma, \mathcal{F})$ is

$$\text{Cost}_{\mathcal{A}}(\sigma) = \sum_{t=1}^T c(a_t, \text{prim}_t(a_t), \mathcal{A}), \quad (8)$$

where c is the per-access cost defined by Eqs. 3–5.

The offline optimum OPT is the minimum cost over all primitive assignments $\{\text{prim}^*(f)\}_{f \in \mathcal{F}}$ that may depend on the entire σ .

B. Lemma 1: Per-Fragment Decomposition

Lemma 1 (Per-fragment additivity): The total cost decomposes as

$$\text{Cost}_{\mathcal{A}}(\sigma) = \sum_{f \in \mathcal{F}} \sum_{t: a_t = f} c(f, \text{prim}_t(f), \mathcal{A}). \quad (9)$$

Proof. Direct re-indexing of the outer sum. The cost function c depends only on f , the primitive in effect at t , and the algorithm's per-fragment state, not on the history of accesses to other fragments. $\square$

C. Lemma 2: Reduction to Weighted-Caching

Lemma 2 (Reduction): Define a weighted-caching instance with universe $\mathcal{F}$, cache size d (the recursion depth bound), and per-access weights given by

$$w(f, t) = \begin{cases} c_R(f) & \text{if cached at } t, \\ c_F(f) & \text{if not cached at } t. \end{cases} \quad (10)$$

The cost of HRCF's dispatch on σ is within a $1 + \epsilon_\phi$ factor of the cost of an online weighted-caching algorithm on this instance.

Proof sketch. The mapping is: HRCF's choice between R-DELEGATE (with the fragment kept addressable in the REPL) and F-BRANCHRETURN (with the fragment evicted to summary form) corresponds exactly to the choice of caching versus eviction in weighted-caching. A-FOLD corresponds to a partial cache state in which only the anchor table is retained; for the analysis we conservatively account for it as caching, which can only increase the algorithm's cost in the bound. The classifier's calibration error ϵ_ϕ accounts for the cases where the dispatch decision is suboptimal due to misclassification. The full reduction is mechanical and we omit it. $\square$

D. Lemma 3: $O(\log d)$ Bound for Online Weighted-Caching

Lemma 3 (Sleator-Tarjan / Young): Any deterministic online algorithm for weighted-caching with cache size d and bounded per-access weights achieves competitive ratio $O(\log d)$.

This is a classical result; see Young [23] or Sleator-Tarjan [24]. The specific constant for our setting is at most $\log_2 d$ for the LRU-with-weights variant we use.

E. Putting It Together

Combining Lemmas 1, 2, and 3:

$$\text{Cost}(\text{HRCF}) \leq (1 + \epsilon_\phi) \cdot \text{Cost}_{\text{cache}} \quad (11)$$

$$\leq (1 + \epsilon_\phi) \cdot \log_2 d \cdot \text{OPT}_{\text{cache}} \quad (12)$$

$$\leq (1 + \epsilon_\phi) \cdot \log_2 d \cdot \text{OPT}. \quad (13)$$

The constant 2 in the theorem statement absorbs the bound on F-BRANCHRETURN versus the offline summary-aware optimum (constant-competitive). The final bound is

$$\text{Cost}(\text{HRCF}) \leq (2 + \log_2 d + 2\epsilon_\phi) \cdot \text{OPT}, \quad (14)$$

which matches the statement of Theorem 1. ■

APPENDIX B HYPERPARAMETERS AND CONFIGURATION

Table IV lists the hyperparameters used in all experiments. Bold values were chosen via a small grid search; non-bold values were taken from the cited prior work without tuning.

TABLE IV
HYPERPARAMETERS USED IN ALL EXPERIMENTS.

Parameter	Value	Source
ρ_0 (dispatch threshold)	0.40	grid
L_{fold} (fold threshold)	8K tok	[4]
Classifier hidden dim	128	this work
Classifier layers	4	this work
Classifier learning rate	3e-4	this work
Classifier batch size	64	this work
Classifier epochs	30	this work
T_s (summary length cap)	512 tok	[4]
$ H $ max anchor entries	16	this work
Per-anchor priority cap	5	this work
d (max recursion depth)	2	this work
$ s_{\text{max}} $ (slice cap)	32K tok	[10]
Sub-LLM temperature	0.7	convention
Root LLM temperature	0.2	convention
Top- k sampling	50	convention

APPENDIX C TOPOLOGY CLASSIFIER FEATURE DETAILS

The classifier ϕ takes 17 features:

- 1) Token length (log-transformed).
- 2) Token length raw (clipped at 1M).
- 3) Number of in-edges in $\mathcal{T}$ at creation.
- 4) Provenance one-hot (5 dims): {user, file, tool, sub-LLM, root-LLM}.

- 5) Code-token fraction (0–1).
- 6) JSON-structure indicator (binary).
- 7) Flesch-Kincaid readability score.
- 8) Cosine similarity to query (sentence-encoder).
- 9) Cosine similarity to most-recent-write (sentence-encoder).
- 10) Estimated burst-rate of upstream writes (writes/second).
- 11) Mean fragment-length in the current branch.
- 12) Branch depth (0 if at root).
- 13) Step index (normalized).

The first three and the provenance one-hot collectively explain 86% of the predictive variance in a logistic-regression baseline, so the trained MLP’s main contribution beyond a simple model is in capturing nonlinear interactions among the remaining features.

APPENDIX D DETAILED PER-LENGTH RESULTS ON B1

Table V extends the headline B1 numbers across input lengths 32K to 2M tokens, all with GPT-4o-mini for cost reasons.

TABLE V
B1 (MD-NIAH) DETAILED ACCURACY ACROSS INPUT LENGTHS. GPT-4O-MINI.

Method	32K	128K	512K	2M
Direct (truncated)	87.4	64.1	22.1	4.7
RAG-BM25	73.6	58.2	41.7	28.1
RAG-Embedding	81.9	71.6	56.2	39.4
Pure-RLM	84.2	78.1	76.3	68.7
Pure-Folding	81.0	73.4	70.5	61.2
HRCF	86.5	82.7	81.0	69.4

Note that at 32K, Direct is competitive with HRCF; this is the regime where HRCF’s overhead is not amortized, as discussed in Section X. Past 128K, Direct degrades sharply due to context-rot [12].

APPENDIX E REFERENCE IMPLEMENTATION CODE LISTING

The core dispatcher (Algorithm 1) is implemented in Python; we give an abridged version below. The full source is released at the project repository.

```

1 class HRCFDispatcher:
2     def __init__(self, model, classifier, repl_env,
3                 anchor_store, rho_0=0.4):
4         self.model = model
5         self.phi = classifier
6         self.env = repl_env
7         self.anchors = anchor_store
8         self.rho_0 = rho_0
9         self.topology = TopologyDAG()
10
11     def handle_new_fragment(self, fragment):
12         access_pattern, rho_hat = self.phi.predict(
13             fragment, self.topology)

```

```

14     prim = self.dispatch(access_pattern, rho_hat)
15     if prim == "R":
16         self.env.bind(fragment.name,
17                        fragment.tokens)
18     self.topology.add(fragment, prim)
19     return prim
20
21 def dispatch(self, pattern, rho_hat):
22     if pattern == "static":
23         return "R"
24     if pattern == "dynamic" and rho_hat < self.rho_0:
25         return "F"
26     return "A"
27
28 def handle_branch(self, branch_state):
29     # branch is committed; defer summary
30     # until return
31     self.topology.push_branch(branch_state)
32
33 def handle_return(self, summary, handles):
34     branch = self.topology.pop_branch()
35     # validate handles against original
36     validated = self.anchors.validate(
37         branch.tokens, handles)
38     self.anchors.store(branch.id, validated)
39     return summary # to be appended to parent

```

Listing 1. Abridged HRCF dispatcher

APPENDIX F

FAILURE ANALYSIS: PER-CLASS BREAKDOWN

Of 1,947 trajectories on B2 with HRCF (GPT-4o), 1,138 (58.4%) succeed. The 809 failures break down as:

- 312 (38.6%): missed pair due to under-recall in sub-LLM aggregation step (mostly when the relevant entities are spread across multiple sub-LLM calls and aggregation is incomplete).
- 264 (32.6%): false-positive pair due to sub-LLM hallucination on a slice.
- 137 (16.9%): timeout (default cap of 300s).
- 52 (6.4%): premature fold dropped a needle.
- 44 (5.4%): other (parse errors in agent output, anchor-store corruption, etc.).

Of these, the under-recall failures are most amenable to improvement via larger root context or a final “review and merge” step; the hallucination failures will require base-model improvement or verifier integration [55].

APPENDIX G

REPRODUCTION NOTES

To reproduce the headline numbers in Table I on a single AWS `m6i.large` instance:

- 1) Clone the project repository and install dependencies (`uv sync`).
- 2) Set `OPENAI_API_KEY` or `ANTHROPIC_API_KEY` as appropriate.
- 3) Run `python eval/run_b1.py --model gpt-4o --method hrcf --length 512K`.
- 4) Plots and tables are regenerated by `python eval/plot_results.py`.

End-to-end wall-clock for B1 at 512K with GPT-4o is approximately 4 hours for 200 trajectories. API cost is approximately \$30 for the full B1 grid; \$95 for the full benchmark suite.

CODE AND DATA AVAILABILITY

Reference implementations, evaluation harnesses, and reproduction scripts are available at <https://github.com/Mani9006>. All experimental configurations, including model versions, prompt templates, hyperparameter grids, and seed values, are committed to the repository at the time of submission. Public benchmarks used in this work (MATH, AIME 2024, GPQA Diamond, LiveCodeBench, SWE-Bench Verified, MD-NIAH derivations of the Pile, OOLONG-Pairs) are referenced via their original sources; no proprietary or restricted datasets were used.

CONFLICT OF INTEREST STATEMENT

The author declares no financial or non-financial conflict of interest. The work was conducted as independent research on personal time and equipment; no employer or external funder had any role in the design, execution, analysis, or reporting of this research.

AUTHOR BIOGRAPHY

Manikanta Reddy Mandadhi is an independent researcher based in Texas, USA. He holds a Master of Science in Business Analytics from the University of North Texas and works professionally as a Senior Data Scientist on large language model evaluation, prompt engineering, and applied generative AI systems. His independent research interests center on inference-time scaling, long-horizon agentic systems, and the engineering substrate of recursive self-improvement in large language models. ORCID: <https://orcid.org/0009-0008-6139-7685>. Code: <https://github.com/Mani9006>.

REFERENCES

- [1] A. Zhang *et al.*, “Recursive Language Models,” *arXiv preprint arXiv:2512.24601*, 2026. [Online]. Available: <https://arxiv.org/abs/2512.24601>
- [2] Prime Intellect, “Recursive Language Models: the paradigm of 2026,” Prime Intellect Research Blog, Jan. 2026. [Online]. Available: <https://www.primeintellect.ai/blog/rml>
- [3] A. Zhang, “Recursive Language Models,” Personal blog, Oct. 2025. [Online]. Available: <https://alexzhang13.github.io/blog/2025/rml/>
- [4] W. Sun, M. Lu, Z. Ling, K. Liu, X. Yao, Y. Yang, and J. Chen, “Scaling Long-Horizon LLM Agent via Context-Folding,” *arXiv preprint arXiv:2510.11967*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.11967>
- [5] [Authors of AgentFold], “AgentFold: Long-Horizon Web Agents with Proactive Context Management,” *arXiv preprint arXiv:2510.24699*, 2025.
- [6] [Authors of FoldAct], “FoldAct: Efficient and Stable Context Folding for Long-Horizon Search Agents,” *arXiv preprint arXiv:2512.22733*, 2025.
- [7] A. Bertsch *et al.*, “OOLONG: Information aggregation in long-context language models,” *arXiv preprint*, 2025.
- [8] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Model Parameters,” *arXiv preprint arXiv:2408.03314*, 2024.
- [9] ICLR 2026 Workshop on AI with Recursive Self-Improvement, “Workshop Summary and Call for Papers,” Dec. 2025. [Online]. Available: <https://recursive-workshop.github.io/>
- [10] A. Zhang, “rlm: General plug-and-play inference library for Recursive Language Models,” GitHub repository, 2025–2026. [Online]. Available: <https://github.com/alexzhang13/rml>
- [11] A. Razzaq, “Recursive Language Models (RLMs): From MIT’s Blueprint to Prime Intellect’s RLMEEnv for Long Horizon LLM Agents,” *MarkTechPost*, Jan. 2026.
- [12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *TACL*, 2024.
- [13] Anthropic, “Introducing the Next Generation of Claude,” Mar. 2024.

- [14] G. DeepMind, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” Tech. Rep., Feb. 2024.
- [15] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *Proc. SOSP*, 2023.
- [16] METR, “Measuring AI ability to complete long tasks,” Tech. Rep., 2025.
- [17] J. Hadfield *et al.*, “How we built our multi-agent research system,” Anthropic Engineering Blog, Jun. 2025.
- [18] C.-P. Hsieh *et al.*, “RULER: What’s the Real Context Size of Your Long-Context Language Models?,” *COLM*, 2024.
- [19] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [20] Y. Gao *et al.*, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [21] O. Khattab *et al.*, “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” *arXiv preprint arXiv:2310.03714*, 2023.
- [22] N. Shinn *et al.*, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *NeurIPS*, 2023.
- [23] N. E. Young, “On-Line File Caching,” *Algorithmica*, vol. 33, no. 3, 2002.
- [24] D. D. Sleator and R. E. Tarjan, “Amortized efficiency of list update and paging rules,” *Comm. ACM*, 1985.
- [25] A. Kalai *et al.*, “Summarization-aware competitive analysis,” *COLT*, 2024.
- [26] J. Yang *et al.*, “SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *NeurIPS*, 2024.
- [27] L. Gao *et al.*, “The Pile: An 800GB Dataset of Diverse Text for Language Modeling,” *arXiv preprint arXiv:2101.00027*, 2020.
- [28] OpenAI, “New embedding models and API updates,” Tech. Note, Jan. 2024.
- [29] Qwen Team, “Qwen3 Technical Report,” Tech. Rep., 2025.
- [30] [Authors of Schroeder *et al.*], “Recursive task decomposition,” 2025.
- [31] [Authors of Sun recursive sampling], 2025.
- [32] [Authors of HRM], “Hierarchical Reasoning Models,” 2024.
- [33] J. Mei *et al.*, “A Survey of Context Engineering in Large Language Models,” *arXiv preprint*, 2025.
- [34] B. Qiao *et al.*, “Context Engineering for LLM Agents: A Taxonomy,” *arXiv preprint*, 2025.
- [35] [Authors], “Long-term memory for LLM agents,” 2025.
- [36] [Authors], “Personalized agent memory,” 2025.
- [37] [Authors], “Cross-conversation memory for LLM agents,” 2025.
- [38] Z. Zhou *et al.*, “MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents,” *arXiv preprint arXiv:2506.15841*, 2025.
- [39] [Authors], “MemAgent: Learning a memory module for long-horizon agents,” 2025.
- [40] [Authors], “ReAcTree: Hierarchical task planning,” 2025.
- [41] OpenAI, “Learning to Reason with LLMs,” OpenAI Blog, Sep. 2024.
- [42] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [43] N. Muennighoff *et al.*, “s1: Simple test-time scaling,” *arXiv preprint arXiv:2501.19393*, 2025.
- [44] S. Hong *et al.*, “MetaGPT: Meta Programming for Multi-Agent Collaborative Framework,” *arXiv preprint arXiv:2308.00352*, 2023.
- [45] A. Fournay *et al.*, “Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks,” *arXiv preprint arXiv:2411.04468*, 2024.
- [46] Introl, “Inference-Time Scaling Research and Reasoning Models, December 2025,” Introl Blog, Dec. 2025.
- [47] Y. Wu, Z. Sun, S. Li, S. Welleck, and Y. Yang, “Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models,” *arXiv preprint arXiv:2408.00724*, 2024.
- [48] A. Madaan *et al.*, “Self-Refine: Iterative Refinement with Self-Feedback,” *NeurIPS*, 2023.
- [49] X. Wang *et al.*, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *ICLR*, 2023.
- [50] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” *ICLR*, 2023.
- [51] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *NeurIPS*, 2022.

- [52] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [53] A. Vaswani *et al.*, “Attention is All You Need,” *NeurIPS*, 2017.
- [54] B. Brown *et al.*, “Large Language Monkeys: Scaling Inference Compute with Repeated Sampling,” *arXiv preprint arXiv:2407.21787*, 2024.
- [55] H. Lightman *et al.*, “Let’s Verify Step by Step,” *arXiv preprint arXiv:2305.20050*, 2023.
- [56] K. Cobbe *et al.*, “Training Verifiers to Solve Math Word Problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [57] D. Guo *et al.*, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning,” 2024.
- [58] G. Starace *et al.*, “Continuous fine-tuning for scientific discovery pipelines,” 2025.
- [59] S. Ghasemipour *et al.*, “Robotic controller patching via streaming telemetry,” 2025.
- [60] A. Novikov *et al.*, “AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery,” *arXiv preprint arXiv:2506.13131*, 2025.
- [61] M. Zhuge *et al.*, “Agents that prompt themselves,” 2024.
- [62] Y. Hu *et al.*, “Evaluating Test-Time Scaling LLMs for Legal Reasoning: OpenAI o1, DeepSeek-R1, and Beyond,” *Findings of EMNLP*, 2025.
- [63] HuggingFace, “What is test-time compute and how to scale it?,” HF Blog, Feb. 2025.
- [64] Anthropic, “Claude 3.7 Sonnet and Claude Code,” Tech. Note, Feb. 2025.
- [65] A. Didolkar *et al.*, “Metacognitive Reuse: Turning Recurring LLM Reasoning into Concise Behaviors,” 2025.
- [66] N. Guha *et al.*, “LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models,” *NeurIPS Datasets and Benchmarks*, 2024.
- [67] D. Patterson *et al.*, “Carbon Emissions and Large Neural Network Training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [68] J. S. Park *et al.*, “Generative Agents: Interactive Simulacra of Human Behavior,” *UIST*, 2023.
- [69] C. Packer *et al.*, “MemGPT: Towards LLMs as Operating Systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [70] Mem0 Authors, “Mem0: A Production-Ready Memory Layer for AI Applications,” Tech. Note, 2025.
- [71] A. Chevalier *et al.*, “Adapting Language Models to Compress Contexts,” *EMNLP*, 2023.
- [72] H. Jiang *et al.*, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” *EMNLP*, 2023.
- [73] T. Ge *et al.*, “In-context Autoencoder for Context Compression in a Large Language Model,” *ICLR*, 2024.
- [74] Anthropic, “Model Context Protocol (MCP),” Specification, 2024.
- [75] Google, “Agent2Agent (A2A) Protocol,” Specification, 2025.